



TRABALHO PRÁTICO - T02

SIMULADOR DE GERÊNCIA DE MEMÓRIA (PAGINAÇÃO)

Formato: Grupos de até 4 alunos

Linguagem: Python (versão 3.10 ou superior)

Ambiente de Entrega: Repositório Público no GitHub

1. INTRODUÇÃO E OBJETIVOS

A gerência de memória virtual é um dos componentes mais críticos de um Sistema Operacional moderno. Conforme abordado no livro-texto (*Operating System Concepts*, 9a edição, Capítulo 9), quando a memória física (RAM) está cheia e um novo processo precisa de páginas de memória, o SO deve decidir qual página existente será substituída. Este trabalho prático tem como objetivo solidificar a compreensão dos conceitos de **Memória Virtual, Paginação e Algoritmos de Substituição de Páginas**. Cada grupo deverá completar um simulador que processe um fluxo de referências a endereços de memória, gerencie uma tabela de páginas (Page Table) e uma quantidade limitada de quadros de memória física (Frames), gerando um mapa visual da memória a cada instrução executada.

2. REPOSITÓRIO BASE E MODELO DE CÓDIGO-FONTE

Vocês não iniciarão o projeto do zero. O professor disponibilizou um esqueleto de código funcional que realiza a leitura do arquivo de entrada, a limpeza de comentários e a estrutura de classes principais (Frame, TabelaPaginas, Simulador).

O código-fonte base deve ser obtido no seguinte endereço:

- **URL do Repositório:** <https://github.com/ProfessorFilipo/MemSim/tree/main>

Os grupos devem clonar ou baixar o arquivo `simulador_memoria.py` e trabalhar estritamente dentro dele, preenchendo as seções de lógica algorítmica e de exibição visual marcadas explicitamente com o comentário `# TODO: IMPLEMENTAR PELO GRUPO`.



3. ESPECIFICAÇÃO DO PROJETO E REQUISITOS

O simulador de cada grupo deverá, obrigatoriamente:

1. **Mapear Acessos:** Processar o número da página solicitado a partir do arquivo de entrada.
2. **Detectar Falhas de Página (Page Faults):** Verificar se a página solicitada está presente nos frames da memória RAM simulada.
3. **Aplicar Algoritmos de Substituição:** Caso a memória física esteja cheia e ocorra uma falha de página, o simulador deve aplicar os algoritmos designados para o grupo para escolher a página "vítima" que sairá da memória.
4. **Atualizar Estatísticas:** Contabilizar o número total de acessos, o total de falhas de página (*page faults*) e calcular a taxa percentual de falhas.
5. **Exibir o Mapa de Memória:** A cada linha processada do arquivo de entrada (passo a passo), o programa deve imprimir no terminal o estado atual da memória física, mostrando quais páginas ocupam quais quadros (*frames*) e indicando visualmente qual frame foi alterado se houve substituição.

4. DISTRIBUIÇÃO DOS ALGORITMOS POR GRUPO (15 COMBINAÇÕES ÚNICAS)

Para garantir a autoria e a diversidade das análises teóricas, a turma está dividida em 15 grupos. Cada grupo possui uma combinação única de **dois algoritmos a serem implementados e comparados**, bem como a **quantidade de frames fixos** a serem configurados no arquivo de teste.

Os algoritmos disponíveis na literatura do livro-texto (Capítulo 9) e mapeados para este trabalho são:

- **FIFO:** First-In, First-Out (Seção 9.4.1)
- **LRU:** Least Recently Used (Seção 9.4.3)
- **Ótimo (OPT):** Optimal Page Replacement (Seção 9.4.2)
- **Segunda Chance (Clock):** Second-Chance/Clock Algorithm (Seção 9.4.4)
- **LFU:** Least Frequently Used (Seção 9.4.5)
- **MFU:** Most Frequently Used (Seção 9.4.5)

Tabela de Atribuição:

- **Grupo 01:** FIFO vs. LRU (com 3 frames)
- **Grupo 02:** FIFO vs. LRU (com 4 frames)
- **Grupo 03:** FIFO vs. Ótimo (OPT) (com 3 frames)



- **Grupo 04:** FIFO vs. Ótimo (OPT) (com 4 frames)
- **Grupo 05:** FIFO vs. Segunda Chance (Clock) (com 3 frames)
- **Grupo 06:** FIFO vs. Segunda Chance (Clock) (com 4 frames)
- **Grupo 07:** LRU vs. Ótimo (OPT) (com 3 frames)
- **Grupo 08:** LRU vs. Ótimo (OPT) (com 4 frames)
- **Grupo 09:** LRU vs. Segunda Chance (Clock) (com 3 frames)
- **Grupo 10:** LRU vs. Segunda Chance (Clock) (com 4 frames)
- **Grupo 11:** Segunda Chance (Clock) vs. Ótimo (OPT) (com 3 frames)
- **Grupo 12:** Segunda Chance (Clock) vs. Ótimo (OPT) (com 4 frames)
- **Grupo 13:** FIFO vs. LFU (com 3 frames)
- **Grupo 14:** FIFO vs. LFU (com 4 frames)
- **Grupo 15:** LRU vs. LFU (com 3 frames)

Nota: O grupo deverá adaptar o código base para aceitar a escolha do algoritmo via parâmetro ou criar duas versões do arquivo executável no repositório final.

5. CASOS DE USO PARA VALIDAÇÃO (GABARITO DE TESTES)

Para que os alunos validem se a lógica programada está correta, abaixo estão os resultados passo a passo esperados para a mesma cadeia de referências utilizando 3 frames.

Conteúdo do arquivo de teste comum (entrada.txt):

Plaintext

3
7
0
1
2
0
3
0
4
2
3
0
3



Caso de Uso 1: Algoritmo FIFO (3 Frames)

Resultado esperado no terminal durante a execução passo a passo:

Plaintext

Iniciando simulação com 3 frames disponíveis.

=====

--- Passo 1: Acesso à Página 7 (Page Fault) ---

[Frame 0]: Página 7 <--- Alterado

[Frame 1]: [Vazio]

[Frame 2]: [Vazio]

--- Passo 2: Acesso à Página 0 (Page Fault) ---

[Frame 0]: Página 7

[Frame 1]: Página 0 <--- Alterado

[Frame 2]: [Vazio]

--- Passo 3: Acesso à Página 1 (Page Fault) ---

[Frame 0]: Página 7

[Frame 1]: Página 0

[Frame 2]: Página 1 <--- Alterado

--- Passo 4: Acesso à Página 2 (Page Fault) ---

[Frame 0]: Página 2 <--- Alterado

[Frame 1]: Página 0

[Frame 2]: Página 1

--- Passo 5: Acesso à Página 0 (Hit) ---

[Frame 0]: Página 2

[Frame 1]: Página 0

[Frame 2]: Página 1



Pontifícia Universidade Católica do Rio Grande do Sul
Escola Politécnica

Sistemas Operacionais – 2026/I
Prof. Filipo Mór – www.filipomor.com



--- Passo 6: Acesso à Página 3 (Page Fault) ---

[Frame 0]: Página 2

[Frame 1]: Página 3 <--- Alterado

[Frame 2]: Página 1

--- Passo 7: Acesso à Página 0 (Page Fault) ---

[Frame 0]: Página 2

[Frame 1]: Página 3

[Frame 2]: Página 0 <--- Alterado

--- Passo 8: Acesso à Página 4 (Page Fault) ---

[Frame 0]: Página 4 <--- Alterado

[Frame 1]: Página 3

[Frame 2]: Página 0

--- Passo 9: Acesso à Página 2 (Page Fault) ---

[Frame 0]: Página 4

[Frame 1]: Página 2 <--- Alterado

[Frame 2]: Página 0

--- Passo 10: Acesso à Página 3 (Page Fault) ---

[Frame 0]: Página 4

[Frame 1]: Página 2

[Frame 2]: Página 3 <--- Alterado

--- Passo 11: Acesso à Página 0 (Page Fault) ---

[Frame 0]: Página 0 <--- Alterado

[Frame 1]: Página 2

[Frame 2]: Página 3

--- Passo 12: Acesso à Página 3 (Hit) ---

[Frame 0]: Página 0

[Frame 1]: Página 2



[Frame 2]: Página 3

===== STATS FINAIS =====

Total de Acessos: 12

Total de Page Faults: 10

Taxa de Page Faults: 83.33%

=====

Caso de Uso 2: Algoritmo LRU (3 Frames)

Resultado esperado no terminal durante a execução passo a passo:

Plaintext

Iniciando simulação com 3 frames disponíveis.

=====

--- Passo 1: Acesso à Página 7 (Page Fault) ---

[Frame 0]: Página 7 <-- Alterado

[Frame 1]: [Vazio]

[Frame 2]: [Vazio]

--- Passo 2: Acesso à Página 0 (Page Fault) ---

[Frame 0]: Página 7

[Frame 1]: Página 0 <-- Alterado

[Frame 2]: [Vazio]

--- Passo 3: Acesso à Página 1 (Page Fault) ---

[Frame 0]: Página 7

[Frame 1]: Página 0

[Frame 2]: Página 1 <-- Alterado

--- Passo 4: Acesso à Página 2 (Page Fault) ---

[Frame 0]: Página 2 <-- Alterado

[Frame 1]: Página 0



[Frame 2]: Página 1

--- Passo 5: Acesso à Página 0 (Hit) ---

[Frame 0]: Página 2

[Frame 1]: Página 0

[Frame 2]: Página 1

--- Passo 6: Acesso à Página 3 (Page Fault) ---

[Frame 0]: Página 2

[Frame 1]: Página 0

[Frame 2]: Página 3 <--- Alterado

--- Passo 7: Acesso à Página 0 (Hit) ---

[Frame 0]: Página 2

[Frame 1]: Página 0

[Frame 2]: Página 3

--- Passo 8: Acesso à Página 4 (Page Fault) ---

[Frame 0]: Página 4 <--- Alterado

[Frame 1]: Página 0

[Frame 2]: Página 3

--- Passo 9: Acesso à Página 2 (Page Fault) ---

[Frame 0]: Página 4

[Frame 1]: Página 0

[Frame 2]: Página 2 <--- Alterado

--- Passo 10: Acesso à Página 3 (Page Fault) ---

[Frame 0]: Página 4

[Frame 1]: Página 3 <--- Alterado

[Frame 2]: Página 2

--- Passo 11: Acesso à Página 0 (Page Fault) ---



[Frame 0]: Página 0 <-- Alterado

[Frame 1]: Página 3

[Frame 2]: Página 2

--- Passo 12: Acesso à Página 3 (Hit) ---

[Frame 0]: Página 0

[Frame 1]: Página 3

[Frame 2]: Página 2

===== STATS FINAIS =====

Total de Acessos: 12

Total de Page Faults: 9

Taxa de Page Faults: 75.00%

=====

Atenção: Os casos de uso acima servem apenas para guiar o desenvolvimento do grupo. No dia da apresentação/entrega, o professor testará o software com arquivos de entrada inéditos contendo tamanhos de memória diferentes e sequências de acesso geradas aleatoriamente.

6. REQUISITOS DE ENTREGA E DIRETRIZES DO GITHUB

A entrega deste trabalho prático seguirá um modelo rigoroso de desenvolvimento de software de mercado:

1. **Repositório Público Único:** Cada grupo deve criar ou utilizar um repositório **público** no GitHub. O link desse repositório deve ser enviado ao professor até a data limite da entrega.
2. **Auditoria de Contribuição Individual (Histórico do Git):** Todos os integrantes do grupo devem clonar o repositório e enviar suas respectivas contribuições (commits e pushes) utilizando suas contas pessoais do GitHub. **O histórico de commits do repositório será auditado pelo professor.** Caso um integrante não possua participações e commits significativos registrados em seu usuário Git no histórico do projeto, ele receberá nota **zero**, independentemente do resultado final do trabalho do grupo.
3. **Código 100% Funcional e Executável:** O código final entregue no branch principal (main ou master) deve estar totalmente finalizado, limpo e livre de bugs. O professor irá clonar o repositório diretamente no terminal de avaliação e executá-lo imediatamente



com o interpretador oficial Python 3. Não serão tolerados erros de importação, caminhos absolutos locais fixados no código (hardcoded) ou dependências externas não documentadas. Se o programa falhar na execução inicial da auditoria, o grupo perderá os pontos referentes à corretude do algoritmo.

7. CRITÉRIOS DE AVALIAÇÃO

A nota final do trabalho prático será composta pelos seguintes critérios:

- **Corretude Teórica (40%):** Implementação exata e correta da lógica matemática e conceitual dos dois algoritmos sorteados, validada através de testes dinâmicos com novos arquivos inseridos pelo professor.
- **Mapa de Memória e Formatação de Saída (20%):** Fidelidade visual do mapa impresso no terminal a cada passo do simulador, incluindo a marcação correta do frame modificado e o resumo estatístico.
- **Qualidade do Código e Versionamento (20%):** Organização do código-fonte, boas práticas de nomenclatura, ausência de redundâncias e uso correto das ferramentas do Git e GitHub de forma distribuída.
- **Defesa do Trabalho (20%):** Entrevista individualizada ou coletiva com o grupo durante a aula de entrega, onde os alunos deverão explicar trechos específicos da lógica implementada e responder a questionamentos baseados no Capítulo 9 do livro-texto.